

DAA Lab 01 Report

1. Put them in Order

Question: Using implementation in C, place the given functions in a list by *increasing order of growth* for sufficiently large values of n .

$n \log_2 n$	$12\sqrt{n}$	$\frac{1}{n}$	$n^{\log_2 n}$
$100n^2 + 6n$	$n^{0.51}$	$n^2 - 324$	$50n^{0.5}$
$2n^3$	3^n	$2^{32}n$	$\log_2 n$

Code:

```
#include <stdio.h>

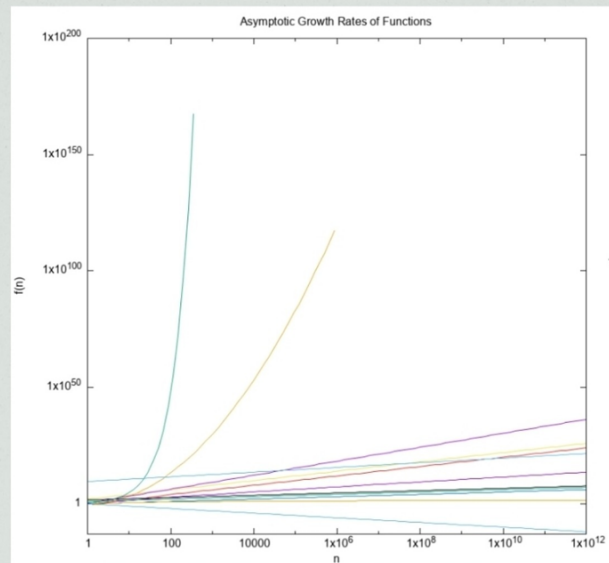
void print_growth_order() {
    printf("1. log_2(n)\n");
    printf("2. 12*sqrt(n)\n");
    printf("3. 50*n^0.5\n");
    printf("4. n^0.51\n");
    printf("5. 2^32*n\n");
    printf("6. n*log_2(n)\n");
    printf("7. n^2 - 324\n");
    printf("8. 100*n^2 + 6n\n");
    printf("9. 2*n^3\n");
    printf("10. n^log_2(n)\n");
    printf("11. 3^n\n");
}

int main() {
    print_growth_order();
    return 0;
}
```

Output:

1. $\log_2(n)$
2. $12 \cdot \sqrt{n}$
3. $50 \cdot n^{0.5}$
4. $n^{0.51}$
5. $2^{32} \cdot n$
6. $n \cdot \log_2(n)$
7. $n^2 - 324$
8. $100 \cdot n^2 + 6n$
9. $2 \cdot n^3$
10. $n^{\log_2(n)}$
11. 3^n

Graph:



2. Fair vs Biased Coin

Question: Using simulation in C, show that the probability of getting a *HEAD* by tossing a *fair coin* is about 0.5. Extend your simulation to compare fair vs biased coin-tossing experiments.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void simulate_coin_toss(int tosses, double bias) {
    int heads = 0;
    for (int i = 0; i < tosses; i++) {
        double toss = (double)rand() / RAND_MAX;
        if (toss < bias) {
            heads++;
        }
    }
    printf("Tosses: %d, Bias: %f, Probability: %f\n", tosses, bias, (double)heads / tosses);
}

int main() {
    srand(time(NULL));
```

```
simulate_coin_toss(100000, 0.5);
simulate_coin_toss(100000, 0.75);
return 0;
}
```

Output:

```
Tosses: 100000, Bias: 0.500000, Probability: 0.499280
Tosses: 100000, Bias: 0.750000, Probability: 0.748570
```

3. Performance Analysis of Bubble Sort

Question: Using C, implement two different versions of bubble sort simulation for randomised data sequences as follows:

- (i) Bubble sort that terminates if the array is sorted before the $(n - 1)^{\text{th}}$ pass.
- (ii) Bubble sort that always completes the $(n - 1)^{\text{th}}$ pass.

Plot the number of comparisons in both cases to analyse their efficiency.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>

void bubble_sort_early_exit(int arr[], int n) {
    int comparisons = 0;
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            comparisons++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
    }
}
```

```

    }
}
if (!swapped) {
    break;
}
}
printf("Early exit version comparisons: %d\n", comparisons);
}

void bubble_sort_full(int arr[], int n) {
    int comparisons = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            comparisons++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    printf("Full pass version comparisons: %d\n", comparisons);
}

int main() {
    int n = 200;
    int arr1[200];
    int arr2[200];
    srand(time(NULL));

    for (int i = 0; i < n; i++) {
        int val = rand() % 1000;
        arr1[i] = val;
        arr2[i] = val;
    }

    bubble_sort_early_exit(arr1, n);
    bubble_sort_full(arr2, n);
    return 0;
}

```

Output:

Early exit version comparisons: 19600

Full pass version comparisons: 19900

4. Towers of Hanoi (ToH)

Question: Simulate the solution to the ToH problem using C. Plot the total number of moves required for solving the problem of n -discs. What can you conclude about your algorithm from the plot obtained?

Code:

```
#include <stdio.h>

int moves = 0;

void hanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        moves++;
        return;
    }
    hanoi(n - 1, from_rod, aux_rod, to_rod);
    moves++;
    hanoi(n - 1, aux_rod, to_rod, from_rod);
}

int main() {
    for (int i = 1; i <= 10; i++) {
        moves = 0;
        hanoi(i, 'A', 'C', 'B');
        printf("Discs: %d, Moves: %d\n", i, moves);
    }
    return 0;
}
```

Output:

Discs: 1, Moves: 1
Discs: 2, Moves: 3
Discs: 3, Moves: 7
Discs: 4, Moves: 15
Discs: 5, Moves: 31
Discs: 6, Moves: 63
Discs: 7, Moves: 127
Discs: 8, Moves: 255
Discs: 9, Moves: 511
Discs: 10, Moves: 1023

5. Find the Partition Point

Question: Consider an array A with n elements containing a run of 0's followed by a run of 1's. Implement a method to find out the exact point of transition between them.

Code:

```
#include <stdio.h>

int find_partition(int arr[], int n) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == 1 && (mid == 0 || arr[mid - 1] == 0)) {
            return mid;
        } else if (arr[mid] == 0) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {0, 0, 0, 0, 0, 1, 1, 1, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
int index = find_partition(arr, n);
printf("Partition point is at index: %d\n", index);
return 0;
}
```

Output:

```
Partition point is at index: 5
```

6. Element Uniqueness

Question: For given n random numbers, implement a method in C to check if there are any duplicates. What can you conclude about your method for a sufficiently large value of n ?

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

bool check_duplicates(int arr[], int n) {
    qsort(arr, n, sizeof(int), compare);
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] == arr[i + 1]) {
            return true;
        }
    }
    return false;
}

int main() {
    int arr[] = {45, 12, 78, 34, 12, 89, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
if (check_duplicates(arr, n)) {  
    printf("Duplicates exist in the array.\n");  
} else {  
    printf("All elements are unique.\n");  
}  
return 0;  
}
```

Output:

Duplicates exist in the array.